

Práctica 2: Syscalls

Syscalls y APIs en un RTOS de misión crítica para un marcapasos VVI

1 Objetivos

1.1 Objetivo general

Diseñar e implementar una capa de abstracción de llamadas al sistema (Syscalls) y servicios de kernel en FreeRTOS sobre el ESP32, aislando la lógica médica del marcapasos VVI del acceso directo al hardware para garantizar la seguridad operativa y el determinismo temporal del sistema.

1.2 Objetivos específicos

- Implementar un despachador de servicios de sistema protegido para mediar las solicitudes entre las tareas de aplicación médica y los recursos del núcleo.
- Desarrollar llamadas al sistema para el sensado ventricular sincronizado y la emisión de pulsos de estimulación con validación de parámetros temporales en el kernel.
- Integrar mecanismos de temporización de alta precisión, registro centralizado de eventos y supervisión del temporizador de vigilancia (Watchdog) a través de la interfaz de servicios del sistema operativo.

2 Investigación previa

1. **Modos de Privilegio:** ¿Cuál es la diferencia fundamental entre el modo usuario y el modo supervisor (kernel) en un sistema operativo? ¿Por qué se considera peligroso que el código de la aplicación de usuario manipule los pines o registros del microcontrolador directamente?
2. **Llamadas al Sistema:** ¿Qué es una *Syscall*? Explique cómo difiere el mecanismo de ejecución de una Syscall respecto a llamar a una función normal en lenguaje C.
3. **Manejo de Interrupciones (ISR):** ¿Qué es una Rutina de Servicio de Interrupción (ISR) en un microcontrolador? Mencione al menos dos operaciones que están estrictamente prohibidas dentro de una ISR (por ejemplo, retardos o impresiones en consola) y justifique por qué.
4. **Primitivas de FreeRTOS:** Investigue qué es una Cola de Mensajes (*Queue*) y qué es un Semáforo Binario en un RTOS. ¿Cómo ayudan estos mecanismos a transferir información de forma segura entre el hardware (ISR) y las tareas del sistema operativo?
5. **Lógica del Marcapasos VVI:** En el contexto de la estimulación cardíaca a demanda, defina brevemente qué es el “Intervalo de Escape” y qué función cumple el “Periodo Refractario Ventricular” para evitar sobre-sensado (falsos positivos).

3 Caso de estudio

De: Dirección de Ingeniería y Desarrollo

Para: Equipo de Desarrollo de Firmware RTOS (R&D)

Fecha: 24 de Agosto de 2026, 08:30 CST

Asunto: Avances semanales y arquitectura de servicios/syscalls para firmware VVI

Hola equipo,

Hablé con el equipo de soporte de plataforma y ya modularizaron el código de arranque y diagnóstico (*bootstrapping*) de la semana pasada. Me comentaron que les enviarán los archivos de cabecera e implementación en el transcurso del día; procuremos mantener en todo momento el código desacoplado y modular en nuestras ramas.

Por otra parte, la División de Investigación Clínica y Sistemas Biomédicos nos solicita avances críticos para esta misma semana en la separación de privilegios del RTOS. Les reenvío íntegro el correo que nos hicieron llegar con los requerimientos técnicos.

Saludos cordiales,

Líder Técnico de Proyecto

—— Mensaje Reenviado ——

De: División de Investigación Clínica y Sistemas Médicos

Para: Dirección de Ingeniería y Desarrollo

Fecha: 23 de Agosto de 2026, 17:15 CST

Asunto: RE: Requerimientos de arquitectura de kernel y servicios para firmware VVI

Estimado equipo de desarrollo,

Para la siguiente etapa de desarrollo del marcapasos VVI sobre el ESP32, requerimos que el equipo establezca formalmente una capa de llamadas al sistema (Syscalls) y servicios de kernel que aisle completamente la lógica médica del acceso directo a los registros de hardware y periféricos.

Es indispensable contar con un mecanismo de despacho concurrente y protegido que atienda las operaciones críticas. El sistema debe ofrecer servicios protegidos para la consulta de tiempo con precisión de microsegundos, la captura y sincronización estricta de las ondas R ventriculares intrínsecas (evitando bloqueos indefinidos o carreras críticas), y la emisión de pulsos de estimulación con validación en el kernel para rechazar duraciones de pulso que se encuentren fuera de la ventana terapéutica segura ($100\ \mu\text{s}$ a $2000\ \mu\text{s}$).

Asimismo, se debe centralizar el registro de telemetría médica en un búfer circular protegido y garantizar la alimentación periódica del Watchdog de hardware exclusivamente a través de la interfaz del supervisor para evitar falsos disparos de seguridad.

Adjuntamos un documento con los detalles técnicos requeridos para que procedan a desglosar los requerimientos funcionales, no funcionales y las restricciones de diseño previas a la implementación.

Atentamente,

Investigación y Desarrollo Biomédico

Documento Adjunto: Especificación Técnica de Servicios de Kernel y Syscalls (VVI-SYS-SPEC-v2.1)

Resumen Ejecutivo del Subsistema de Servicios

El objetivo del subsistema es establecer una barrera de contención de fallos entre las tareas de control médico (espacio de aplicación) y los periféricos de estimulación, sensado y temporización (espacio de kernel) en el microcontrolador ESP32 bajo FreeRTOS.

1. Contexto Operativo del Marcapasos VVI

El marcapasos en modo VVI opera en el ventrículo y responde bajo una lógica de estimulación a demanda:

- **Intervalo de Escape Ventricular (T_{escape}):** Frecuencia base garantizada de 60 ppm ($T_{\text{escape}} = 1000$ ms). Si no se detecta actividad intrínseca en este intervalo, se debe forzar una estimulación ventricular.
- **Periodo Refractario Ventricular (VRP):** Intervalo posterior a un sensado o estimulación ($VRP = 250$ ms) durante el cual el sistema ignora cualquier ruido eléctrico para evitar sobre-sensado de la onda T.
- **Ventana de Sensado Activa:** Periodo de escucha activa equivalente a $T_{\text{escape}} - VRP = 750$ ms.

2. Requerimientos de la Capa de Syscalls y Servicios

ID Servicio	Servicio Solicitado	Criterios Técnicos y Restricciones
SYS_GET_TIME	Temporización de alta resolución	Retorno de tiempo monótonico en microsegundos (μs) sin desbordamiento temprano. Debe ser seguro ante accesos concurrentes.
SYS_WAIT_SENSE	Sincronización de sensado ventricular	Bloqueo eficiente de la tarea médica en espera de una despolarización ventricular (onda R). La notificación debe originarse en la ISR asociada al pin de sensado sin introducir latencias mayores a $50 \mu s$.
SYS_PACE_PULSE	Generación de pulso de estimulación	Validación obligatoria en espacio de kernel: rechazar pulsos con ancho $< 100 \mu s$ o $> 2000 \mu s$. La generación física del pulso debe ser atómica y no interrumpible por tareas de menor prioridad.
SYS_LOG_EVENT	Registro y telemetría	Transferencia de cadenas de texto informativas y marcas temporales hacia el puerto serie o búfer circular sin bloquear la ejecución de la tarea médica.
SYS_KICK_WDT	Refresco del Watchdog	Alimentación del temporizador de vigilancia (<i>Task WDT</i>) ejecutada exclusivamente por el hilo supervisor tras validar el estado saludable de la aplicación.

[illegible]

2. Delimitación del Modelo de privilegio (espacio de usuario vs. espacio de kernel):

- ¿Por qué la tarea médica (`vvi_controller_task`) no debe acceder directamente a funciones de bajo nivel como `digitalWrite()`, `attachInterrupt()` o registros directos del microcontrolador?

- Determine qué componentes deben ser de visibilidad estrictamente interna dentro del archivo de implementación del kernel (`syscalls.cpp`) y cuáles deben exponerse a través de la interfaz de cabecera pública (`syscalls.h`).

3. Mecanismo de despacho y comunicación inter-contextos:

- Dado que el ESP32 ejecuta FreeRTOS sobre una arquitectura donde el cambio de modo de hardware puede ser emulado mediante primitivas de sincronización, justifique el uso de una **cola de solicitudes de syscalls** (*Syscall Request Queue*) frente a llamadas a funciones compartidas directas.

- ¿Qué estructura de datos (`struct`) permite empaquetar de forma genérica el identificador de la llamada, los parámetros variables y el canal de retorno del resultado?

- ¿Qué primitiva de sincronización (semáforo binario, cola o notificación) es la más adecuada para bloquear a la tarea solicitante hasta que el supervisor del kernel complete la operación y retorne un resultado?

4. Sincronización de eventos asíncronos (sensado ventricular):

- El evento de sensado de la onda R ventricular ocurre de manera asíncrona mediante una interrupción externa (ISR). ¿Por qué no es seguro realizar operaciones bloqueantes o procesamiento pesado dentro de la rutina de interrupción?

- Analice el flujo de sincronización: ¿Cómo debe interactuar la ISR con la tarea médica para que la llamada `sys_wait_sensing()` se desbloquee inmediatamente al detectar la despolarización intrínseca sin consumir ciclos innecesarios de CPU?

5. Validación defensiva y temporización en kernel:

- Establezca los criterios de validación que debe aplicar el despachador de kernel al recibir una solicitud de estimulación (`sys_pace_pulse`). Si la aplicación solicita un ancho de pulso fuera de la norma clínica ($< 100 \mu s$ o $> 2000 \mu s$), ¿cuál debe ser la respuesta del kernel para proteger al paciente sin colapsar el sistema operativo?

-
-
-
-
-
-
-
-
- Analice la relación entre el hilo supervisor y el Watchdog de hardware configurado en el arranque: ¿quién debe ser el responsable formal de alimentar el temporizador de vigilancia y bajo qué condiciones?
-
-
-
-
-
-
-
-

4.2 Implementación en plataforma y simulación

Para la simulación del sistema, cargue en el entorno Wokwi la topología de hardware mediante el archivo `diagram.json` provisto a continuación. El pin GPIO 4 operará como entrada de sensado simulando la despolarización intrínseca (onda R), mientras que el GPIO 5 funcionará como salida de estimulación conectado al LED rojo.

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "
      attrs": {} },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 73.4,
      "left": -96,
      "attrs": { "color": "yellow", "bounce": "0" }
    },
    { "type": "wokwi-led", "id": "led1", "top": -90, "left": 99.8, "attrs": { "
      color": "red" } },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 3.4,
      "left": 95.15,
      "rotate": 270,
      "attrs": { "value": "220" }
    }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "btn1:1.1", "esp:4", "yellow", [ "h-28.8", "v144", "h249.6", "v-86.4" ] ],
    [ "esp:5", "r1:1", "cyan", [ "h24.04", "v-129.55" ] ],
```

```

[ "r1:2", "led1:A", "cyan", [ "h0" ] ],
[ "esp:GND.1", "btn1:2.1", "black", [ "h-119.81", "v-48" ] ],
[ "esp:GND.2", "led1:C", "black", [ "v0" ] ]
],
"dependencies": {}
}

```

Código 1: diagram.json

Con base en su análisis arquitectónico, el firmware debe dividirse estrictamente en la capa de servicios (syscalls.h y syscalls.cpp), el módulo de arranque (bootstrap) y la lógica de aplicación médica (main.cpp). Utilice las siguientes plantillas para implementar el despachador de kernel y la tarea de control, completando la lógica requerida en las secciones indicadas con la etiqueta TODO.

```

static void kernel_service_task(void *pvParameters) {
    esp_task_wdt_add(NULL);
    syscall_req_t req;

    while (1) {
        if (xQueueReceive(syscall_queue, &req, portMAX_DELAY) == pdTRUE) {
            switch (req.id) {
                case SYS_GET_TIME_US:
                    req.result.time_us = (uint64_t)esp_timer_get_time();
                    break;

                case SYS_WAIT_SENSING:
                    // TODO: Evaluar el semáforo de sensado con el timeout solicitado.
                    // Asignar el resultado (true/false) a req.result.success.
                    break;

                case SYS_PACE_PULSE:
                    // TODO: Implementar validación estricta en espacio de kernel.
                    // Rechazar pulsos con ancho menor a 100 us o mayor a 2000 us.
                    // Si el ancho es válido, ejecutar el pulso físico en
                    PIN_VENTRICLE_PACE
                    // y asignar req.result.success = true; de lo contrario, false.
                    break;

                case SYS_LOG_EVENT:
                    Serial.printf("[KERNEL LOG @ %lluus] %s\r\n",
                        (unsigned long long)esp_timer_get_time(), req.args.
log_msg);
                    break;

                case SYS_KICK_WATCHDOG:
                    // TODO: Refrescar el temporizador de vigilancia de la tarea actual.
                    break;

                default:
                    break;
            }

            if (req.done_sem != NULL) {
                xSemaphoreGive(req.done_sem);
            }
        }
    }
}

```

Código 2: syscalls.cpp (fragmento del despachador)

```
#include "bootstrap.h"
#include "syscalls.h"

#define ESCAPE_INTERVAL_MS 1000 // Ritmo base garantizado de 60 ppm
#define REFRACTORY_PERIOD_MS 250 // Periodo refractario ventricular (VRP)

void vvi_controller_task(void *pvParameters) {
    sys_log_event("Controlador VVI iniciado.");

    while (1) {
        sys_kick_watchdog();

        // TODO: Calcular la ventana de escucha activa basándose en
        // el intervalo de escape y el periodo refractario.
        uint32_t listen_window = 0; // Reemplazar con el cálculo correcto

        // TODO: Consumir la API del sistema para bloquear la tarea en espera
        // de una onda R durante la ventana de escucha calculada.
        bool sensed = false; // Reemplazar con la llamada a sys_wait_sensing

        if (sensed) {
            sys_log_event("Onda R detectada. Estimulacion inhibida.");
        } else {
            sys_log_event("Escape agotado. Estimulando ventriculo...");
            // TODO: Consumir la API del sistema para emitir un pulso de 500 us.
        }

        // TODO: Consumir la API del sistema (sys_sleep_ms) para suspender
        // la tarea durante el periodo refractario (VRP) e ignorar ruido.
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    bootstrap_init();
    sys_init();

    xTaskCreatePinnedToCore(vvi_controller_task, "VVI_Task", 4096, NULL, 2, NULL,
        0);
}

void loop() {
    vTaskDelete(NULL);
}
```

Código 3: main.ino (aplicación principal)

4.3 Pruebas y validación del sistema

Puede realizar esta validación utilizando el simulador Wokwi o, si ya dispone de su hardware, implementándolo directamente en su placa ESP32 (se otorgará puntaje adicional por realizar las pruebas en físico). Ejecute los siguientes escenarios de control clínico. Si el sistema no responde como se

describe, analice el flujo de sincronización y los tiempos de bloqueo en `syscalls.cpp`, aplique las correcciones necesarias y repita la prueba.

1. Escenario 1: Estimulación por escape (modo base)

Acción: Inicie el sistema (en simulador o hardware) y deje correr el control sin presionar el botón.

Esperado: La terminal debe imprimir un texto similar a "Escape agotado. Estimulando ventriculo..." con una separación exacta de 1000 ms entre eventos. El temporizador de vigilancia (Watchdog) no debe disparar reinicios y el LED de estimulación debe destellar brevemente (500 μ s) en cada ciclo.

2. Escenario 2: Inhibición por sensado intrínseco

Acción: Presione el botón aproximadamente medio segundo después del último pulso.

Esperado: El sistema debe reaccionar de inmediato imprimiendo "Onda R detectada. Estimulacion inhibida.". El LED de estimulación no debe encender, y el contador de 1000 ms del intervalo de escape debe reiniciarse en ese preciso instante.

3. Escenario 3: Inmunidad en periodo refractario (VRP)

Acción: Inmediatamente después de ver el mensaje de estimulación o sensado en la terminal, presione el botón repetidamente y lo más rápido posible.

Esperado: El RTOS debe ignorar cualquier pulsación física que ocurra dentro de los 250 ms posteriores a un evento. Esto simula el rechazo de ruido o el sobre-sensado de la onda T, por lo que el ritmo base del marcapasos no debe verse alterado.

4. Escenario 4: Validación defensiva de la syscall

Acción: Modifique temporalmente su código en `main.cpp` para solicitar un ancho de pulso fuera de norma, por ejemplo `sys_pace_pulse(3000)` o `sys_pace_pulse(50)`.

Esperado: El despachador de kernel debe evaluar la petición, rechazarla silenciosamente y retornar falso. El LED no debe encender en ningún momento, protegiendo así el tejido cardíaco simulado.

Reporte de Evidencias y Solución de Fallos

Documente en su reporte el resultado de cada escenario. Agregue capturas de pantalla de la terminal serie (o fotografías de su montaje en caso de usar hardware físico) enfocándose en las marcas de tiempo de alta resolución (KERNEL LOG @ ... μ s) para comprobar el determinismo de su solución. Si detectó comportamientos anómalos durante las pruebas iniciales, describa brevemente la causa raíz del fallo y qué líneas de código modificó en el kernel o en la aplicación para solucionarlo.

Evaluación de la Práctica

Instrucciones: Indique su grado de acuerdo con cada una de las siguientes afirmaciones utilizando una escala de 1 a 5, donde:

1. Totalmente en desacuerdo
2. En desacuerdo
3. Neutral
4. De acuerdo
5. Totalmente de acuerdo

Criterio / Afirmación	1	2	3	4	5
Los conceptos y habilidades desarrollados en esta práctica son relevantes para mi formación profesional.					
El trabajo realizado me permitió comprender de forma práctica la teoría revisada en clase.					
Las actividades y problemáticas planteadas durante la sesión resultaron atractivas y motivadoras.					
El enfoque práctico y los retos propuestos captaron mi interés a lo largo del desarrollo.					
El nivel de complejidad técnica fue adecuado respecto a los conocimientos y recursos previos disponibles.					
La claridad de la guía permitió realizar la práctica de manera fluida y sin bloqueos innecesarios.					